Lab 5 - Linear Classifiers & Multiclass SVM

From Theory to Vectorized Implementation

Dataset

CIFAR-10 (10 classes, 50k training images, 10k test images)

Core Concepts

- Support Vector Machines (SVM) & Hinge Loss
- Vectorization via Numpy Broadcasting
- Stochastic Gradient Descent (SGD)
- Hyperparameter Tuning (Grid Search)

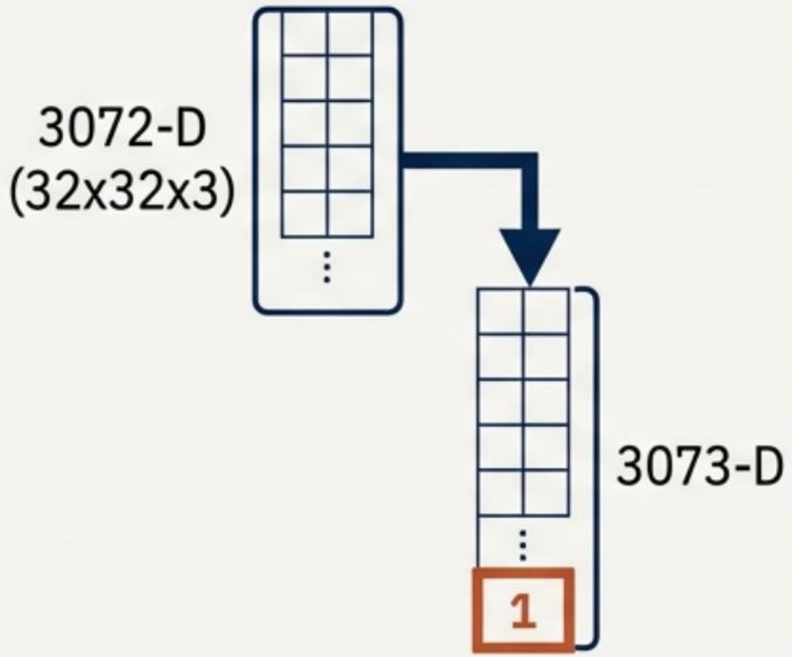


Lab Objectives & Pipeline Overview

The 5 Main Objectives

1. Understand Linear Classifiers & the Bias Trick.
2. Implement Multiclass SVM (Loss & Gradients).
3. Master Vectorized Operations (Eliminate for-loops).
4. Implement SGD Optimization (Mini-batch data).
5. Hyperparameter Tuning (Using the validation set).

The Bias Trick (data_utils.py)



Simplifies **f(x,W) = Wx + b** entirely to

$$f(\mathbf{x}, \mathbf{W}) = \mathbf{W}\mathbf{x}$$

Code Structure Overview:

- `classifiers.py` : Build loss functions and gradients.
- `hyperparameter_tuning.py` : Write grid search logic.

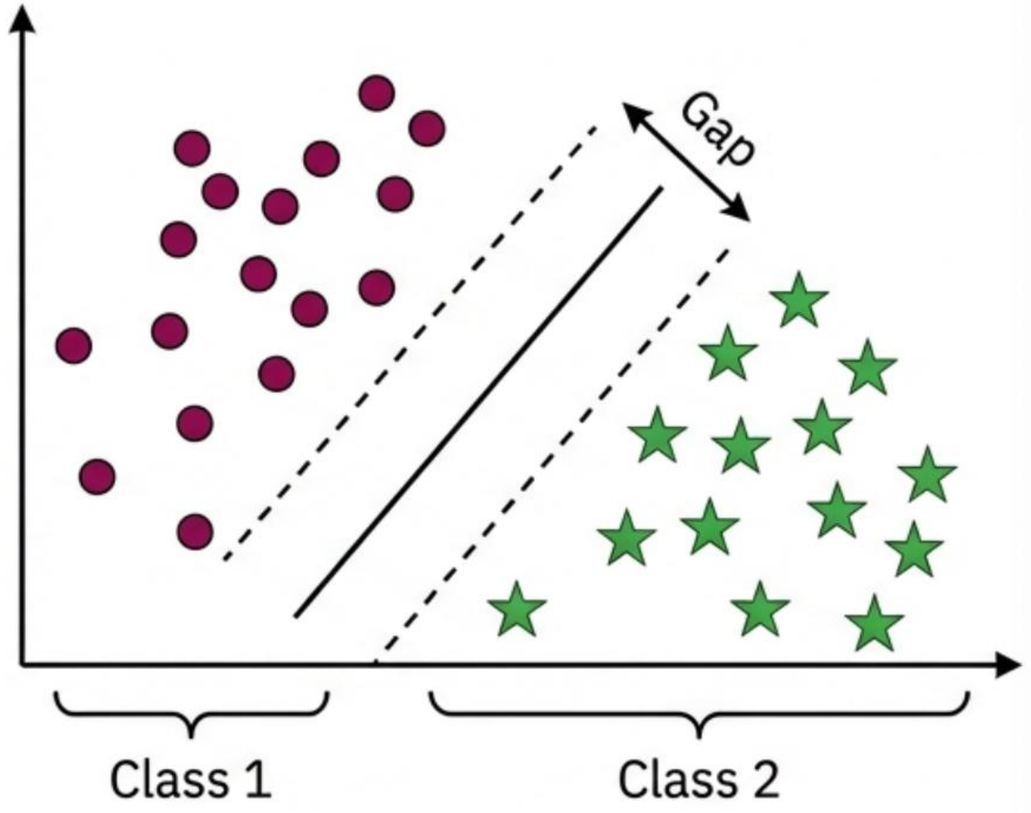
Recap - Multiclass SVM (Hinge Loss)

$$L_i = \sum_{j \neq y_i} \max(0, W_j^T x_i - W_{y_i}^T x_i + \Delta)$$

(Assume Δ = 1 for this lab)

Understanding the Margin

- We only care if the incorrect class score is higher than the correct class score minus the margin.
- If the correct class score is greater than the incorrect score by at least Δ, the loss is 0.



$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda \sum_k ||W_k||^2$$

Don't forget to add L2 Regularization to your final average loss!

TODO 1 - Naive SVM Loss & Gradient

Target File: `svm_loss_naive` inside `classifiers.py`

The Task

- Implement the loss and gradient using explicit for loops.
- Iterate over N samples.
- Iterate over C classes.

Goal: Get the math right first.
Don't worry about speed yet.

Gradient Update Rules

(Applies only when Margin > 0)

Incorrect classes (j):

$$\frac{\partial L_i}{\partial W_j} \mathrel{+}= x_i$$

Correct class (y_i):

$$\frac{\partial L_i}{\partial W_{y_i}} \mathrel{-}= x_i$$

TODO 2 & 3 - The Power of Vectorization

Target File: `svm_loss_vectorized` inside `classifiers.py`

The Task

- **Goal**: Eliminate ALL explicit for loops using Numpy broadcasting.
- Compute all scores for all samples at once: `scores = X.dot(W)`
- Compute margins for all classes simultaneously: `margins = np.maximum(0, scores - correct_class_scores + 1)`
- **Note:** Set the margin of the correct class to 0!
- **Gradient Masking**: Create a binary mask (**margins > 0**) to sum counts.

Expected Outcome: Vectorized code should be 10x to 100x faster than the naive implementation.

```python
def L_i_vectorized(x, y, W):
    scores = W.dot(x)
    margins = np.maximum(0, scores - scores[y] + 1)
    margins[y] = 0
    loss_i = np.sum(margins)
    return loss_i
```

Sanity Check - Debugging the Loss & Gradient

Verify your math before you train!

1. Initial Loss Check

- At initialization, weights W ≈ 0.
- Scores for all classes will be roughly equal.

Expected Initial Loss:
C - 1 = 10 - 1 = 9.0



2. Gradient Check

- Both grad_naive and grad_vectorized compute the exact same mathematical value.
- Calculate the difference:

```
Fira Code
difference = np.linalg.norm(
grad_naive - grad_vectorized)
```

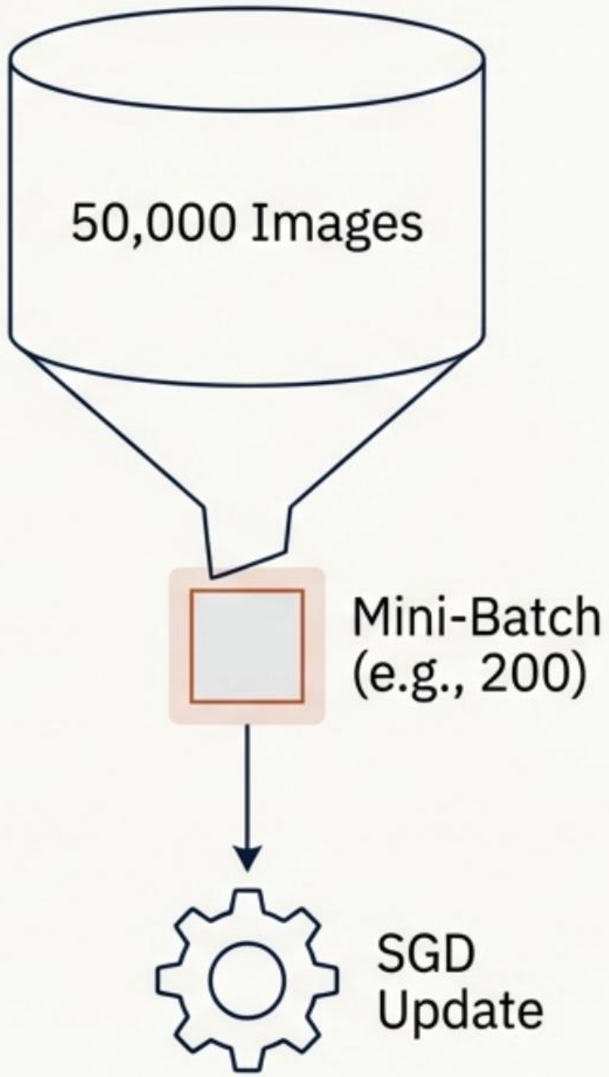
Difference must be < 1e-9

TODO 4 - Stochastic Gradient Descent (SGD)

Target File: `train` method in `LinearClassifier`

The Concept

Computing the gradient over all 50,000 images is too slow.
We use mini-batches.



Implementation Hint

```
Use np.random.choice to randomly sample
batch_size indices.
```

Crucial: Set `replace=True` to sample with replacement.

Weight Update Rule

$$W \leftarrow W - \text{learning_rate} \times \text{gradient}$$

We step in the opposite direction of the gradient to minimize the loss.

TODO 5 - Hyperparameter Tuning (Grid Search)

Target File: `hyperparameter_tuning.py`

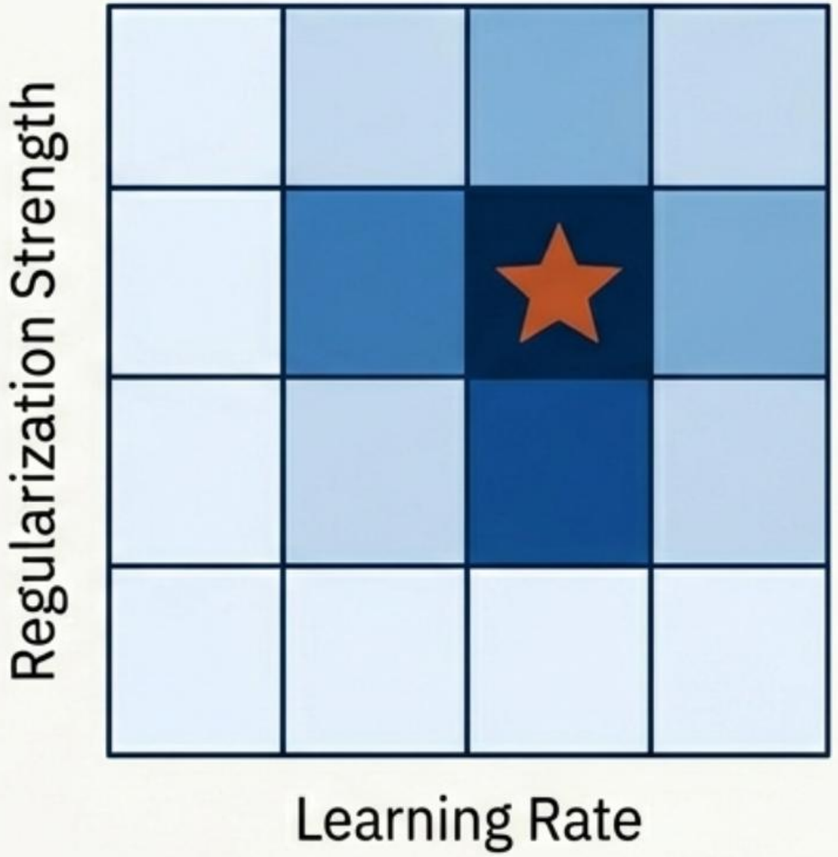
The Golden Rule: NEVER tune your model on the test set. Always use the **validation set.**

The Task

- Write nested loops iterating over combinations of `learning_rates` and `regularization_strengths`.
- Train a new `LinearClassifier` for each combination.
- Evaluate on the validation set and save the best model.

Performance Ceiling:

- A linear classifier operating on raw pixels is limited.
- Expect your best validation/test accuracy to cap out around ~35-40%.



Submission Requirements & Grading



Deliverable 1: Code (60%)

- Completed `classifiers.py` and `hyperparameter_tuning.py`.
- Checked for correctness, SGD logic, and vectorization speedups.

Deliverable 2: Report (40%)

- File: `report_YOURNAME.pdf` (3-5 pages)
- Required contents: Execution logs showing the 10-100x speedup, training loss curves, and hyperparameter tuning tables.
- Analysis Requirements: Explain why vectorization is faster, discuss hyperparameter impacts, and explain the mathematical benefit of the Bias Trick.

TA’s Tips for Success



Start Early

Grid search takes significant time to run. Don’t leave it to the last night!



Test Small

Use a small subset of the data while debugging your loops. Run the full 1500 iterations only when you know it works.



Follow the Rules

Do not alter code outside the designated # TODO blocks.



Academic Integrity

Discuss concepts freely, but write your own Numpy code.

Questions? Look at the course forum, check office hours, or ask now!